



Today's agenda

↳ Basic Calculator 2 → Infix

↳ Postfix

↳ Prefix



AlgoPrep



Q) Basic Calculator 2 → {infix evaluation}

↳ Given a string s which represents an expression, evaluate this expression and return its value.

$$\text{Ex: } 2 + 3 * 4 - 6 + (2 + 3 * 2) = 16$$

//idea 1 → {incorrect idea}

$$2 + 3 * 4 - 6 + (2 + 3 * 2)$$

$$5 + 5 \div 5 \rightarrow 6$$

Bracket → 2

* / % → 1

+ / - → 0

//idea 2

↳ obs 1: we will try to solve the previous operators when we reach a certain operator.

↳ obs 2: for + / -, you can operate left and right separately.

$\text{Post}(st.top) \geq \text{Pre}(i)$
 ↳ solve stack
 operators

$$2 + 3 * 4 - 6 + (2 + 3 * 2)$$

$8+8=16$
 ~~$2+6=8$~~
 ~~$3*2=6$~~
~~8~~
~~8~~
~~2~~
 ~~$14-6=8$~~
~~6~~
 ~~$2+12=14$~~
~~12~~
~~14~~
~~8~~
~~2~~

operand

~~*~~
~~+~~
~~-~~
~~+~~
~~*~~
~~*~~
~~+~~

operator

AlgoPrep



IP Sued's code

```

public int calc (int a, int b, char c) {
    if (c == '+') { return a+b; }
    else if (c == '-') { return a-b; }
    else if (c == '*') { return a*b; }
    else { return a/b; }
}

```

```

int infinevaluation (String s) {

```

```

    Stack<Integer> operand;

```

```

    Stack<Character> operator;

```

```

    int i=0;

```

```

    while (i < s.length()) {

```

```

        if (s[i] >= '0' && s[i] <= '9') {

```

```

            int num=0;

```

```

            while (i < s.length() && s[i] >= '0' && s[i] <= '9') {

```

```

                num = num*10 + (s[i] - '0');

```

```

            }
            operand.push(num);

```

```

        }
        i++;

```

```

    } else if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {

```

```

        while (operator.size() > 0 && prec(operator.peek()) >= prec(s[i])) {

```

```

            char oper = operator.pop();

```

```

            int val2 = operand.pop();

```

```

            int val1 = operand.pop();

```

```

            int cal = calc(val1, val2, oper);
            operand.push(cal);

```

T.C: $O(2N) = O(N)$

S.C: $O(N)$



```
3 operators.push(s[i]);
```

```
3 else {
```

```
3
```

```
i++;
```

```
3
```

```
while (operators.size() > 0) {
```

```
char oper = operators.pop();
```

```
int val2 = operand.pop();
```

```
int val1 = operand.pop();
```

```
int cal = Calc(val1, val2, oper);
```

```
operand.push(cal);
```

```
3
```

```
return operand.pop();
```

```
}
```

```
int prec (char c) {
```

```
if (c == '*' || c == '/') { return 1; }
```

```
else if (c == '+' || c == '-') { return 0; }
```

```
}
```

* , ÷ → 1
+ , - → 0



S: 2 0 4 9 + 3 * 4 - 6

num = 3

Stack < Integer > operand;
Stack < Character > operator;

int i = 0;

while (i < s.length()) {

if (s[i] >= '0' && s[i] <= '9') {

int num = 0;

while (i < s.length() && s[i] >= '0' && s[i] <= '9') {

num = num * 10 + (s[i] - '0');

operand.push(num);

else if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {

while (operator.size() > 0 && Precedence(operator.peek()) >= Precedence(s[i])) {

char op = operator.pop();

int val2 = operand.pop();

int val1 = operand.pop();

int cal = Calc(val1, val2, op);

operand.push(cal);

operator.push(s[i]);

i++;

}

}

}

while (operator.size() > 0) {

char op = operator.pop();

int val2 = operand.pop();

int val1 = operand.pop();

int cal = Calc(val1, val2, op);

operand.push(cal);

}

operand

operator

2061 - 6 = 2055

6

2049 + 12 = 2061

3 * 4 = 12

val2 = 6

val1 = 2061

12

3

2049

←
*
*

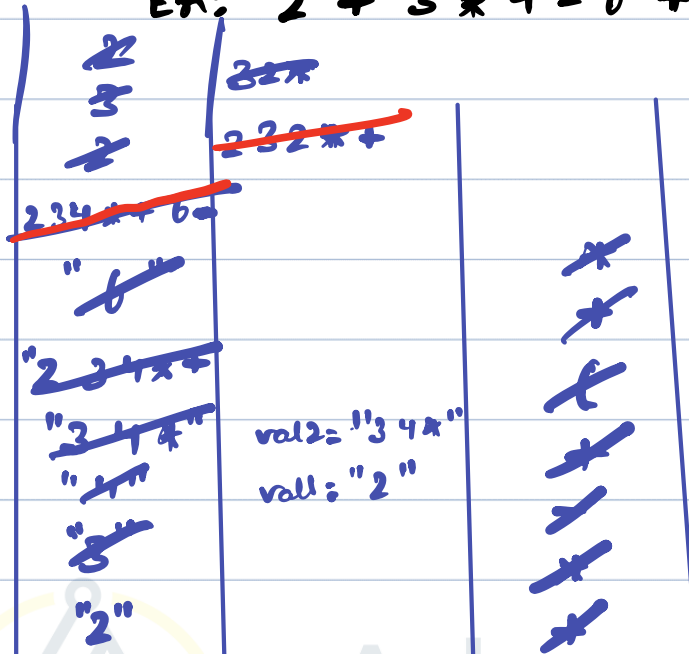
AlgoPrep

Break till 9:50 PM



Q) Infix to Postfix

Ex: $2 + 3 * 4 - 6 + (2 + 3 * 2)$



2 3 4 * + 6 - 2 3 2 * + +

val2 = "3 4 *"
val1 = "2"

operand

operator

AlgoPrep



Q) Infix to Postfix

Ex: $2 + 3 * 4 - 6 + (2 + 3 * 2)$ ⁱ

~~+ 2 * 3 2~~

~~* 3 2~~

2

8

2

~~- + 2 * 3 1 6~~

6

~~+ 2 * 3 4~~

~~* 3 4~~

4

8

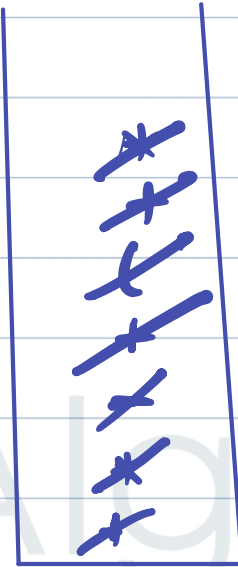
2

val2:
val1:

+ - + 2 * 3 4 6 + 2 * 3 2



operand



operator



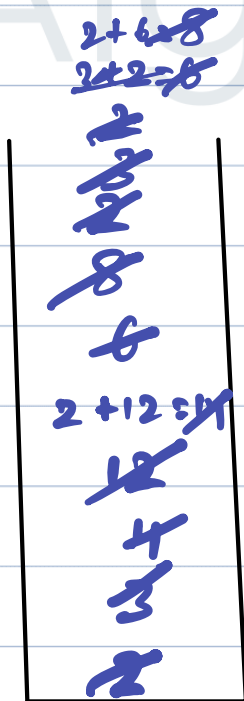
Q) Postfix evaluation

2 3 4 * + 6 - 2 3 2 * + +

obs 1: Precedence has been taken care of.

obs 2: We need 2 operands to execute operator.

2 3 4 * + 6 - 2 3 2 * + +

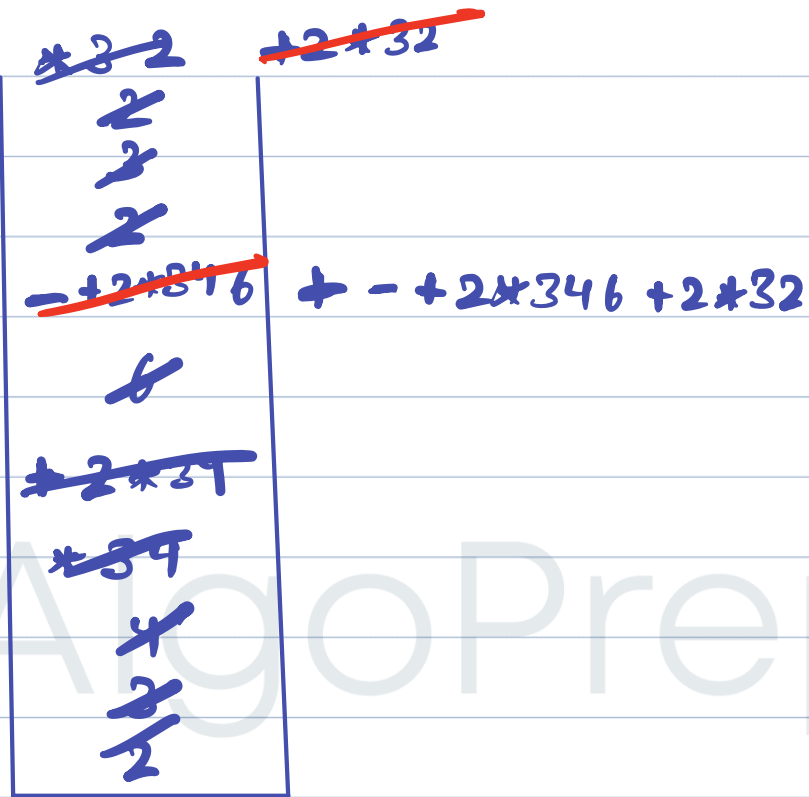


$$8 + 8 = 16$$



Q) Postfix to Prefix

2 3 4 * + 6 - 2 3 2 * + +





Q) Postfin to Infin
↳ HW



AlgoPrep



Q) Postfix evaluation

$\downarrow i$
 $+ - + 2 * 3 4 6 + 2 * 3 2$

$8 + 8 = 16$
$14 - 6 = 8$
$2 + 12 = 14$
2
$3 * 4 = 12$
3
4
6
$2 + 6 = 8$
2
$3 * 2 = 6$
8
2



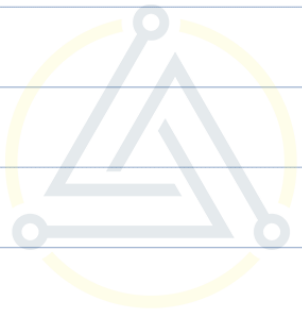
Q) Prefin to in fin
↳ HW



AlgoPrep



Q) Prejin to Postjin



AlgoPrep



Q)

open $\rightarrow \{ +, *, -, \dots \}$ +2 = 2

$$\frac{10}{+} \frac{5}{+} \frac{3}{+} = \text{tan}$$

$10 + 5 + 3$
 $10 * 5 + 3$

$$10 + 5 * 3 \rightarrow (10 + 5) * 3$$
$$\rightarrow 10 + (5 * 3)$$

$$10 * 5 + 3 \rightarrow (10 * 5) + 3$$
$$\rightarrow 10 * (5 + 3)$$



AlgoPrep